



DADOS DE IDENTIFICAÇÃO:

DISCIPLINA: *Programação Orientada a Objetos*

CÓDIGO: 4611F

CRÉDITOS: 4

CARGA HORÁRIA: 60 horas-aula

VALIDADE: a partir de 2025/I

EMENTA:

Desenvolvimento de sistemas de software de complexidade média utilizando o paradigma de orientação a objetos. Estudo e análise dos conceitos de abstração, herança e polimorfismo. Discussão de aspectos avançados sobre classes. Uso de bibliotecas de coleções. Projeto de sistemas orientados a objetos. Manipulação básica de arquivos texto. Noções de programação orientada a eventos.

OBJETIVOS DE APRENDIZAGEM/COMPETÊNCIAS:

O cumprimento da disciplina busca dar ao aluno, ao final do semestre, condições de:

1. Conhecer e utilizar de forma precisa conceitos e termos relacionados ao paradigma de orientação a objetos.
2. Desenvolver as competências e habilidades para a criação de sistemas de complexidade média, formado por múltiplos componentes, e expressar estas soluções na forma de um sistema de classes em uma linguagem de programação.
3. Descrever sistemas utilizando diagramas e código.
4. Construir abstrações para tipos de dados, usando os conceitos de classe, objeto-mensagem, herança e interface.
5. Compreender os conceitos envolvidos em programação orientada a eventos por meio de interface de usuário gráfica.

CONTEÚDOS:

Nº DA UNIDADE: 1

Nº DE HORAS EM PERCENTUAL: 25%

CONTEÚDO: Orientação a objetos básica

1.1. Motivação

1.2. Conceitos de orientação a objetos



1.2.1. Classes e objetos

1.2.2. Atributos e métodos: classe e instância

1.2.3. Tipos de dados: referência e valor

1.3. Visibilidade de atributos e métodos

1.4. Sobrecarga de métodos

1.5. Construtores e destrutores

1.6. Objetos compostos e referências

Nº DA UNIDADE: 2

Nº DE HORAS EM PERCENTUAL: 40%

CONTEÚDO: Orientação a objetos avançada

2.1. Projeto de sistemas Orientados a Objetos

2.1.1. Conceitos básicos

2.1.2. Descrição de sistemas com diagrama de classes UML

2.2. Associação e dependência

2.3. Parametrização de tipos

2.4. Generalização/especialização

2.4.1 Herança simples e múltipla

2.4.2 Hierarquia de classes

2.4.3 Sobrescrita de métodos

2.4.4 Classe abstrata

2.4.5 Interface

2.5. Polimorfismo

2.6. Tratamento de exceções

2.7. Fluxos de dados

Nº DA UNIDADE: 3

Nº DE HORAS EM PERCENTUAL: 20%

CONTEÚDO: Coleções

3.1. Estruturas encadeadas

3.2. Coleções

3.3. Expressões lambda e predicados

Nº DA UNIDADE: 4

Nº DE HORAS EM PERCENTUAL: 15%

CONTEÚDO: Interface de usuário gráfica

4.1. O *framework* de interface de usuário

4.2. Eventos e ações

4.3. Componentes e *containers*

PROCEDIMENTOS METODOLÓGICOS:

- Geral - enfatizar o trabalho em equipe, com a divisão de tarefas e o uso de um repositório comum. Simular situações de compreensão de projeto e manutenção de sistemas. Utilizar ferramenta CASE para associar abstrações de projeto e a implementação. É importante, ao longo do desenvolvimento de todos os exemplos, enfatizar a aplicação de técnicas de depuração de programas. Estabelecer relações claras com a disciplina de Algoritmos e Estrutura de Dados sempre que pertinente. Apresentar o processo de compilação baseado em linha de comando; os alunos devem ser capazes de compilar e executar um programa sem o auxílio de uma IDE.
- Unidade 1 - revisar os conceitos básicos de orientação a objetos e destacar sua utilização em uma linguagem de programação (no caso Java). Trabalhar, em especial, os aspectos de modelagem do conjunto de classes que irá compor a solução de um problema.
- Unidade 2 - aprofundar o entendimento dos conceitos de orientação a objetos abordados na unidade anterior e apresentar conceitos mais avançados sobre o tema. Sugere-se o uso intensivo de cenários e casos em que estes conceitos agreguem valor quando aplicados na resolução de problemas. Seguindo a abordagem da unidade anterior, deve-se mostrar como estes conceitos são utilizados em uma linguagem de programação. Em projeto OO tratar de conceitos básicos como modularidade, coesão e acoplamento, bem como explorar diagrama de classes UML para representar classes, objetos e os relacionamentos entre estes. O uso de ferramenta CASE pode apoiar o desenvolvimento destes tópicos.
- Unidade 3 - enfatizar o uso de coleções para a resolução de problemas (que tipo de coleção é mais adequada para cada tipo de problema), relacionando com aspectos de implementação. As coleções a serem trabalhadas são (pelo menos) listas (com destaque para iteradores), filas, pilhas, mapas e conjuntos.
- Unidade 4 - estudar interfaces gráficas não é explorar extensivamente uma API de GUI do Java (ou outro pacote de interfaces), mas sim explorar três conceitos: o uso de sistemas com interface gráfica (ou seja, sair do console), programação orientada a eventos e um padrão de separação interface-lógica (MVC ou uma variação deste, como no caso do Java).

AVALIAÇÃO:

$$G1 = \frac{P1 + 2.P2 + 2.T + E}{6}$$

6

Sendo que:

P1 – conteúdos até 2.4.3

P2 – conteúdos até a unidade 4.

T – desenvolvimento de estudo de caso.

E – exercícios realizados ao longo do semestre.

Observações:

No que tange aos exercícios, sugere-se que o grau de complexidade deles acompanhe a evolução dos conteúdos abordados ao longo da disciplina e que sejam preferencialmente realizados individualmente.

BIBLIOGRAFIA:

• BÁSICA:

1. HORSTMANN, C. **Java for everyone**: late objects. 2nd ed. Danvers: Wiley, 2013. 589 p.
2. RAMNATH, S.; DATHAN, B. **Object-oriented analysis and design**. New York: Springer, 2010. 440 p.
3. WEISFELD, M. **The object-oriented thought process**. 4th ed. Upper Saddle River: Addison Wesley, 2013. 336 p.

• COMPLEMENTAR:

1. BAESENS, B.; BACKIEL, A.; BROUCKE, S. **Beginning java programming**: the object-oriented approach. Somerset: John Wiley & Sons, 2017. (e-book).
2. DEITEL, P.; DEITEL, H. **Java**: como programar. 10^a ed. São Paulo: Pearson, 2017. (e-book).

3. FARRELL, J. **An object-oriented approach to programming logic and design**. 4th ed. Boston: Cengage Learning, 2013. 560 p.
4. GOODRICH, M. T.; TAMASSIA, R. **Estruturas de dados e algoritmos em Java**. 5^a ed. Porto Alegre: Bookman, 2012. (e-book).
5. LARMAN, Craig. **Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo**. 3^a ed. Porto Alegre: Bookman, 2007. (e-book).

OUTRAS REFERÊNCIAS:

1. Documentação:

- <https://docs.oracle.com/en/java/javase>
- <https://www.jetbrains.com/idea>
- <https://code.visualstudio.com>

2. Sites:

- <https://www.oracle.com/java/technologies/java-se-glance.html>
- <http://docjar.com>

SOFTWARE DE APOIO:

1. Sistema operacional Windows ou Linux
2. Java Platform, Standard Edition (versão mais atual)
3. IDE Visual Studio Code, Eclipse ou IntelliJ (versão mais atual)
4. CASE Astah Professional (<http://astah.net>) ou Modelio (<https://www.modelio.org>)